

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf\_27df53c8-8e0\agent-aa4ece4d73ce087f4.jsonl`
- SHA-256: `93eebf000ebe6c03dd783409b5e420cdf2f305f9a0ba922c3f09a93eafd37667`
- Source modified: `2026-06-10T16:31:53+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf\_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`

Transcript

1. user (2026-06-10T16:28:23.871Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: HDFC parser persists all rows even when the parse is UNRECONCILED, and `txns list` never filters by document status
SEVERITY CLAIMED: high
FILE: src/muneem/parsers/hdfc.py (line 435-458)
DESCRIPTION: Unlike AxisParser/AUParser (which store nothing on an unverified parse), HDFCParse.parse_and_insert inserts every transaction into hdfc_transactions BEFORE computing the verdict and never removes them when the verdict is 'unreconciled' ? the status is recorded but the suspect rows stay. The only read path, `muneem txns list hdfc` (cli.py:347-361 via db.list_hdfc_transactions, db.py:437-440), does not join documents or filter/annotate by status, so rows from a known-bad parse (e.g. a page whose 'Narration'/'Closing' header line failed to extract ? _table_body returns [] for that page and silently drops the whole page's transactions) are served indistinguishably from trusted rows. Related: _parse_transaction line 219 fabricates `closing\_balance=0.0` when the balance token fails _AMOUNT_RE, persisting an invented zero balance on such rows.
EVIDENCE QUOTED: hdfc.py:435-447: `for t in stmt.transactions:
dbmod.insert_hdfc_transaction(...)` runs unconditionally; :454 `verdict = Confidence.RECONCILED
if ok else Confidence.UNRECONCILED` only sets status. db.py:437-440: `SELECT * FROM
hdfc_transactions ORDER BY id DESC LIMIT ?` ? no status filter. hdfc.py:219:
`closing\_balance=closing if closing is not None else 0.0`.
SUGGESTED FIX: Apply the same gate as Axis/AU: compute the verdict first (parse + summary are both pure reads) and insert rows only when reconciled ? or at minimum exclude/flag rows whose document status != 'reconciled' in list queries. Replace the fabricated 0.0 closing balance with None or a parse rejection.

Your job is to try to REFUTE it. Your lens: reachability and impact: check whether the scenario can actually occur in realistic single-user use on Windows, and whether the assigned severity is justified.

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").

- Do not use WebSearch/WebFetch and do not send any repo content to any external service.
Structured output only.

2. user (2026-06-10T16:28:30.232Z)

```
1      """HDFC savings / transaction-account statement parser.
2
3      **Approach:** coordinate / column clustering on **pdfplumber (MIT)** ? not the
4      default ``extract_tables()`` and not a ready-made library. HDFC's transaction
5      table is *semi-ruled*, where ``extract_tables()`` is unreliable; the deep-research
6      decision (see the ``hdfc-parser-tech-decision`` memory) is to cluster words by
7      position. pdfplumber's whole chain is permissive and it opens password-protected
8      statements directly.
9
10     **Layout.** Each transaction is one *primary* line ? the transaction date at the
11     far left (``x0?52``) plus three right-aligned amount columns (Withdrawal, Deposit,
12     Closing Balance) ? followed by zero or more *continuation* lines that wrap the
13     narration and carry the value date (``dd/mm/yyyy`` at ``x0>110``) and the
14     cheque/reference number. We group words into visual lines by ``top``, detect
15     primary lines by a leading date + a closing-balance amount, attach continuations,
16     and read columns by x-position.
17
18     **Correctness is checked, not assumed.** ``reconcile`` verifies the running
19     balance (closing[i] == closing[i-1] - withdrawal + deposit) and
20     ``validate_against_summary`` cross-checks the parse against HDFC's own end-of-
21     statement SUMMARY (Dr/Cr counts + opening/closing balance). Together they prove a
22     parse complete without inspecting any individual value ? which is how we validate
23     against real (sensitive) statements. No statement content is logged here.
24     """
25
26     from __future__ import annotations
27
28     import csv
29     import re
30     from dataclasses import dataclass, field
31     from pathlib import Path
32
33     import pdfplumber
34
35     from muneem import __version__
36     from muneem import db as dbmod
37     from muneem.errors import ParseError
38     from muneem.redaction import redact_filename
39
40     from . import register_parser
41     from .base import StatementParser
42     from .concepts import ConceptRow, StatementConcepts
43     from .profiles import HDFC_PROFILE
44     from .validation import (
45         BookendReport,
46         Confidence,
47         ReconcileReport,
48         reconcile_running_balance,
49         verify,
50     )
51
52     # --- Column geometry (PDF points; A4-portrait HDFC detailed statement) -----
53     # Left-aligned columns key on x0; right-aligned amount columns key on their right
54     # edge (x1), which is stable regardless of the number's magnitude.
55     _DATE_MAX_X0 = 70.0 # transaction date sits at x0?52; the value date is at x0>110
56     _NARR_MIN_X0 = 100.0 # narration band starts at x0?111
57     _AMOUNT_MIN_X0 = 295.0 # amount columns start at x0?306; narration never crosses ?290
58     _WITHDRAWAL_MAX_X1 = 400.0 # withdrawal right edge ?358
59     _DEPOSIT_MAX_X1 = 490.0 # deposit right edge ?443; closing balance right edge ?561
60     _LINE_TOL = 2.5 # words within this many points of each other share a visual line
```

```

61     _CONT_MAX_SPAN = 70.0 # a continuation line is at most this far below its primary line
62
63     _DATE_RE = re.compile(r"^d{2}/d{2}/d{4}$")
64     # Indian-format money: optional minus, comma groups, exactly two decimals, optional
65     Cr/Dr.
66     _AMOUNT_RE = re.compile(r"^-[?[\d,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
67     _FOOTER_MARKERS = {"STATEMENT", "SUMMARY"}
68
69     @dataclass
70     class Transaction:
71         date: str # dd/mm/yyyy, as printed
72         narration: str
73         withdrawal: float
74         deposit: float
75         closing_balance: float
76         value_date: str | None = None
77         ref: str | None = None
78         page: int = 0
79
80         def to_concept(self) -> ConceptRow:
81             """Map HDFC's native row onto the bank-agnostic concept vocabulary (the
82 verifier's input).
83
84             Withdrawal ? ``debit``, deposit ? ``credit``, closing balance ? ``balance``. The
85             ``page``
86             field is HDFC-internal provenance and has no concept equivalent.
87             """
88             return ConceptRow(
89                 txn_date=self.date,
90                 narration=self.narration,
91                 debit=self.withdrawal,
92                 credit=self.deposit,
93                 balance=self.closing_balance,
94                 value_date=self.value_date,
95                 reference=self.ref,
96             )
97
98     @dataclass
99     class Statement:
100         path: Path
101         transactions: list[Transaction] = field(default_factory=list)
102         account_number: str | None = None
103
104     # HDFC's SUMMARY block and its cross-check are now expressed in the shared concept
105     vocabulary +
106     # verifier (the concept-core refactor). These back-compat names keep HDFC's public
107     surface stable:
108     # StatementSummary == the statement-level concepts a SUMMARY block supplies
109     # ValidationReport == the verifier's bookend report (per-row walk + printed-figure
110     checks)
111     StatementSummary = StatementConcepts
112     ValidationReport = BookendReport
113
114     def _to_amount(token: str) -> float | None:
115         """Parse an HDFC money token to a float, or None if it is not money.
116
117         Unlike base.safe_amount we keep ``0.00`` as ``0.0`` (not None): a zero amount
118         is real information and the reconciliation math needs it.
119         """
120         if not _AMOUNT_RE.match(token):
121             return None

```

```

120         cleaned = re.sub(r"(?i)(cr|dr)$", "", token).replace(",", "")
121     try:
122         return float(cleaned)
123     except ValueError:
124         return None
125
126
127 def _group_lines(words: list[dict]) -> list[list[dict]]:
128     """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter)."""
129     lines: list[list[dict]] = []
130     anchor_top: float | None = None
131     for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
132         if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= _LINE_TOL:
133             lines[-1].append(w)
134         else:
135             lines.append([w])
136             anchor_top = w["top"]
137     return [sorted(line, key=lambda w: w["x0"]) for line in lines]
138
139
140 def _line_top(line: list[dict]) -> float:
141     return min(w["top"] for w in line)
142
143
144 def _has_narration(line: list[dict]) -> bool:
145     return any(_NARR_MIN_X0 <= w["x0"] < _AMOUNT_MIN_X0 for w in line)
146
147
148 def _has_amount(line: list[dict]) -> bool:
149     return any(w["x0"] >= _AMOUNT_MIN_X0 and _AMOUNT_RE.match(w["text"]) for w in line)
150
151
152 def _has_leading_date(line: list[dict]) -> bool:
153     first = line[0]
154     return bool(_DATE_RE.match(first["text"]) and first["x0"] < _DATE_MAX_X0)
155
156
157 def _has_footer_marker(line: list[dict]) -> bool:
158     return any(w["text"].upper() in _FOOTER_MARKERS for w in line)
159
160
161 def _is_primary(line: list[dict]) -> bool:
162     """A transaction's first line: a date at the far left + a closing-balance amount.
163
164     Requiring the closing-balance amount guards against stray dates in the page
165     header / account-summary box (e.g. the statement period).
166     """
167     if not _has_leading_date(line):
168         return False
169     return any(
170         w["x0"] >= _AMOUNT_MIN_X0 and w["x1"] >= _DEPOSIT_MAX_X1 and
171         _AMOUNT_RE.match(w["text"])
172         for w in line
173     )
174
175 def _is_continuation(line: list[dict]) -> bool:
176     """A narration-wrap line: narration-band text, no leading date, no amounts."""
177     return _has_narration(line) and not _has_leading_date(line) and not
178     _has_amount(line)
179
180 def _parse_transaction(
181     primary: list[dict], continuations: list[list[dict]], page_no: int
182 ) -> Transaction:

```

```

183 withdrawal = deposit = 0.0
184 closing: float | None = None
185 narr: list[tuple[float, float, str]] = [] # (top, x0, text) for ordered reassembly
186
187 for line in (primary, *continuations):
188     for w in line:
189         x0, x1, text = w["x0"], w["x1"], w["text"]
190         if x0 >= _AMOUNT_MIN_X0:
191             val = _to_amount(text)
192             if val is None:
193                 continue
194             if x1 < _WITHDRAWAL_MAX_X1:
195                 withdrawal = val
196             elif x1 < _DEPOSIT_MAX_X1:
197                 deposit = val
198             else:
199                 closing = val
200         elif x0 >= _NARR_MIN_X0:
201             narr.append((w["top"], x0, text))
202
203 value_date: str | None = None
204 parts: list[str] = []
205 for _top, _x0, text in sorted(narr):
206     if value_date is None and _DATE_RE.match(text):
207         value_date = text # the value date, lifted out of the narration band
208         continue
209     parts.append(text)
210
211 ref_candidates = [p for p in parts if p.isdigit() and len(p) >= 9]
212 ref = max(ref_candidates, key=len) if ref_candidates else None
213
214 return Transaction(
215     date=primary[0]["text"],
216     narration=" ".join(parts),
217     withdrawal=withdrawal,
218     deposit=deposit,
219     closing_balance=closing if closing is not None else 0.0,
220     value_date=value_date,
221     ref=ref,
222     page=page_no,
223 )
224
225
226 def extract_transactions_from_lines(lines: list[list[dict]], page_no: int = 0) ->
list[Transaction]:
227     """Pure, PDF-free core: turn grouped table-body lines into transactions.
228
229     Kept separate from PDF I/O so the column logic is unit-testable on synthetic
230     word lists, with no real data.
231     """
232     primaries = [i for i, line in enumerate(lines) if _is_primary(line)]
233     transactions: list[Transaction] = []
234     for k, idx in enumerate(primaries):
235         stop = primaries[k + 1] if k + 1 < len(primaries) else len(lines)
236         primary = lines[idx]
237         primary_top = _line_top(primary)
238         continuations: list[list[dict]] = []
239         for m in range(idx + 1, stop):
240             line = lines[m]
241             if _has_footer_marker(line) or _line_top(line) - primary_top >
_CONT_MAX_SPAN:
242                 break
243             if _is_continuation(line):
244                 continuations.append(line)
245             transactions.append(_parse_transaction(primary, continuations, page_no))

```

```

246         return transactions
247
248
249     def _table_body(lines: list[list[dict]]) -> list[list[dict]]:
250         """Lines below the column header (drops the per-page account-summary box)."""
251         header_top: float | None = None
252         for line in lines:
253             texts = {w["text"] for w in line}
254             if "Narration" in texts and any(t.startswith("Closing") for t in texts):
255                 header_top = _line_top(line)
256                 break
257         if header_top is None:
258             return []
259         return [line for line in lines if _line_top(line) > header_top + 1]
260
261
262     def _extract_account_number(text: str) -> str | None:
263         """Account number via the HDFC profile (bank knowledge as data; same pattern as
264         before)."""
265         return HDFC_PROFILE.first_account(text)
266
267
268     def parse_statement(pdf_path: Path | str, password: str = "") -> Statement:
269         """Parse an HDFC statement PDF into a :class:`Statement` of transactions.
270
271         ``password`` is forwarded to pdfplumber for encrypted statements and used
272         in-memory only ? never stored or logged.
273         """
274         path = Path(pdf_path)
275         transactions: list[Transaction] = []
276         account_number: str | None = None
277         with pdfplumber.open(str(path), password=password or "") as pdf:
278             for page_no, page in enumerate(pdf.pages):
279                 if account_number is None and page_no < 2:
280                     account_number = _extract_account_number(page.extract_text() or "")
281                 words = page.extract_words(use_text_flow=False, keep_blank_chars=False)
282                 body = _table_body(_group_lines(words))
283                 transactions.extend(extract_transactions_from_lines(body, page_no))
284         return Statement(path=path, transactions=transactions,
285         account_number=account_number)
286
287
288     def reconcile(statement: Statement, tol: float = 0.01) -> ReconcileReport:
289         """Check the running balance via the shared reconciler (closing[i] ==
290         closing[i-1] - withdrawal + deposit).
291
292         Spans page breaks (transactions are in document order), so a clean report also
293         validates page stitching. Reports counts only ? no amounts.
294         """
295         entries = [(t.withdrawal, t.deposit, t.closing_balance) for t in
296         statement.transactions]
297         return reconcile_running_balance(entries, tol)
298
299
300     def _parse_summary_lines(lines: list[str]) -> StatementSummary | None:
301         """Pure core: read HDFC's SUMMARY block from extracted text lines.
302
303         Layout (stable across statements)::
304
305             SUMMARY
306             Opening Balance  Debit Amount  Credit Amount  Closing Balance
307             <opening>        <debitAmt>    <creditAmt>    <closing>
308             Debit Count  Credit Count
309             <debitCount> <creditCount>

```

```

308     The values land on their own lines, so we take the first post-SUMMARY line
309     carrying ?4 money tokens (the balances) and the first carrying ?2 bare
310     integers and no money (the counts).
311     """
312     start = next((i for i, ln in enumerate(lines) if ln.strip() == "SUMMARY"), None)
313     if start is None:
314         return None
315     balances: list[float] | None = None
316     counts: list[int] | None = None
317     for ln in lines[start + 1 : start + 8]:
318         tokens = ln.split()
319         monies = [v for v in (_to_amount(t) for t in tokens) if v is not None]
320         ints = [int(t) for t in tokens if t.isdigit()]
321         if balances is None and len(monies) >= 4:
322             balances = monies[:4]
323         elif counts is None and not monies and len(ints) >= 2:
324             counts = ints[:2]
325     if balances is None and counts is None:
326         return None
327     bvals: list[float | None] = [*balances] if balances else [None, None, None, None]
328     opening, debit_amt, credit_amt, closing = bvals[:4]
329     cvals: list[int | None] = [*counts] if counts else [None, None]
330     debit_count, credit_count = cvals[:2]
331     return StatementSummary(
332         opening_balance=opening,
333         closing_balance=closing,
334         debit_count=debit_count,
335         credit_count=credit_count,
336         debit_amount=debit_amt,
337         credit_amount=credit_amt,
338     )
339
340
341 def parse_summary(pdf_path: Path | str, password: str = "") -> StatementSummary | None:
342     """Read the bank's own SUMMARY figures from the statement, or None if absent."""
343     with pdfplumber.open(str(Path(pdf_path)), password=password or "") as pdf:
344         for page in pdf.pages:
345             text = page.extract_text() or ""
346             if "SUMMARY" in text:
347                 summary = _parse_summary_lines(text.splitlines())
348                 if summary is not None:
349                     return summary
350     return None
351
352
353 def validate_against_summary(
354     statement: Statement, summary: StatementSummary, tol: float = 0.01
355 ) -> ValidationReport:
356     """Cross-check a parse against the bank's SUMMARY: reconciliation + bookends.
357
358     Maps HDFC's rows onto the canonical concepts and delegates to the shared verifier
359     (:func:`muneem.parsers.validation.verify`): reconciliation catches any mid-statement
360     error/omission, the Dr/Cr counts catch any extra/missing row, and the opening &
361     closing
362     balances catch a missing first/last row.
363     """
364     rows = [t.to_concept() for t in statement.transactions]
365     return verify(rows, summary, tol)
366
367
368 def write_csv(statement: Statement, out_path: Path | str) -> Path:
369     """Write parsed transactions to CSV for local QA (the 'export to a sheet' idea).
370
371     Writes real data ? call only for a local, gitignored destination.
372     """

```

```

372 out = Path(out_path)
373 out.parent.mkdir(parents=True, exist_ok=True)
374 with open(out, "w", newline="", encoding="utf-8") as fh:
375     writer = csv.writer(fh)
376     writer.writerow(
377         [
378             "date",
379             "value_date",
380             "narration",
381             "ref",
382             "withdrawal",
383             "deposit",
384             "closing_balance",
385             "page",
386         ]
387     )
388     for t in statement.transactions:
389         writer.writerow(
390             [
391                 t.date,
392                 t.value_date or "",
393                 t.narration,
394                 t.ref or "",
395                 f"{t.withdrawal:.2f}",
396                 f"{t.deposit:.2f}",
397                 f"{t.closing_balance:.2f}",
398                 t.page,
399             ]
400         )
401     return out
402
403
404 @register_parser
405 class HDFCParser(StatementParser):
406     """Registered parser for HDFC savings / transaction-account statements."""
407
408     def can_handle(self, text: str, sender: str | None = None, subject: str | None =
409 None) -> bool:
410         low = (text or "").lower()
411         is_hdfc = "hdfc bank" in low or "hdfcbank" in low or "hdfcbank" in (sender or
412 "").lower()
413         is_statement = "savings account details" in low or (
414             "narration" in low and "closing balance" in low
415         )
416         return is_hdfc and is_statement
417
418     def parse_and_insert(
419         self,
420         pdf_path: Path,
421         conn,
422         doc_id: int,
423         password: str | None = None,
424     ) -> int:
425         """Parse, insert rows into ``hdfc_transactions``, and record parse quality.
426
427         Sets ``documents.status`` to 'reconciled' / 'unreconciled' based on the
428         balance walk (and the bank's SUMMARY when present) so a downstream consumer
429         can tell a trustworthy parse from a suspect one. Returns the row count.
430         """
431         try:
432             stmt = parse_statement(pdf_path, password=password or "")
433         except ParseError:
434             raise
435         except Exception as exc:
436             raise ParseError(f"failed to parse HDFC statement

```

```

{redact_filename(pdf_path)}}") from exc
435         for t in stmt.transactions:
436             dbmod.insert_hdfc_transaction(
437                 conn,
438                 document_id=doc_id,
439                 account_number=stmt.account_number,
440                 txn_date=t.date,
441                 value_date=t.value_date,
442                 particulars=t.narration,
443                 ref=t.ref,
444                 withdrawal=t.withdrawal,
445                 deposit=t.deposit,
446                 closing_balance=t.closing_balance,
447             )
448         summary = parse_summary(pdf_path, password=password or "")
449         ok = (
450             validate_against_summary(stmt, summary).ok
451             if summary is not None
452             else reconcile(stmt).ok
453         )
454         verdict = Confidence.RECONCILED if ok else Confidence.UNRECONCILED
455         dbmod.record_parse(
456             conn, doc_id, confidence=verdict.value, parser="HDFCParser",
457             parser_version=__version__
458         )
459         return len(stmt.transactions)

```

3. user (2026-06-10T16:28:31.122Z)

```

380         """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
381         parse)."""
382         conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence,
383         document_id))
384         record_provenance(
385             conn,
386             document_id=document_id,
387             source_type=source_type,
388             parser=parser,
389             parser_version=parser_version,
390             mapping_layer=mapping_layer,
391             confidence=confidence,
392         )
393
394     def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
395         return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
396         (document_id,)).fetchone()
397
398     def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
399         return _insert_row(conn, "offers", fields)
400
401     def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
402         return conn.execute(
403             """
404             SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
405             d.external_id, d.path
406             FROM offers o
407             JOIN documents d ON d.id = o.document_id
408             ORDER BY o.id
409             """
410         ).fetchall()

```

```

412
413     def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
414         return _insert_row(conn, "axis_transactions", fields)
415
416
417     def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
418         return _insert_row(conn, "axis_cc_transactions", fields)
419
420
421     def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
422         return conn.execute(
423             "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
424         ).fetchall()
425
426
427     def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
428         return conn.execute(
429             "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
430         ).fetchall()
431
432
433     def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
434         return _insert_row(conn, "hdfc_transactions", fields)
435
436
437     def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
438         return conn.execute(
439             "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
440         ).fetchall()
441
442
443     def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
444         return _insert_row(conn, "au_transactions", fields)
445
446
447     def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
448         return conn.execute(
449             "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
450         ).fetchall()
451

```

4. assistant (2026-06-10T16:28:37.733Z)

Now let me check the CLI read path and compare with the Axis/AU parsers.

5. user (2026-06-10T16:28:39.426Z)

```

300
301
302     @offers_app.command("list")
303     def offers_list(
304         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
305     ) -> None:
306         """List extracted offers."""
307         from muneem import db as dbmod
308
309         db_path = db or load_config().db_path
310         if not Path(db_path).exists():
311             typer.echo("no offers yet (database not created).")
312             return

```

```

313
314     with _ledger(db_path) as conn:
315         dbmod.init_db(conn)
316         rows = dbmod.list_offers(conn)
317
318     if not rows:
319         typer.echo("no offers found.")
320         return
321     for row in rows:
322         who = row["merchant"] or row["external_id"] or "?"
323         when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
324         tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
325         typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")
326
327
328     @txns_app.command("list")
329     def txns_list(
330         bank: str = typer.Argument(..., help="Which bank's transactions to list (e.g. hdfc,
axis)."),
331         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
332         limit: int = typer.Option(50, help="Max number of transactions to show."),
333     ) -> None:
334         """List parsed transactions for a specific bank."""
335         from muneem import db as dbmod
336
337         db_path = db or load_config().db_path
338         if not Path(db_path).exists():
339             typer.echo("database not created yet.")
340             return
341
342         bank = bank.lower()
343         if bank not in BANK_DISPATCH:
344             typer.echo(f"Unknown bank: {bank}", err=True)
345             return
346
347         list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
348         with _ledger(db_path) as conn:
349             dbmod.init_db(conn)
350             rows = list_fn(conn, limit=limit)
351
352         if not rows:
353             typer.echo(f"no {bank} transactions found.")
354             return
355         for row in rows:
356             if bank == "axis_cc":
357                 amount = f"+{row[pos_key]}" if row["type"] == "Cr" else f"-{row[neg_key]}"
358             else:
359                 amount = f"+{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
360             part = (row["particulars"] or "")[:50]
361             typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}")
362
363
364     @db_app.command("status")
365     def db_status(
366         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
367     ) -> None:
368         """Show the ledger's encryption state (no statement data is read or printed)."""
369         from muneem import db as dbmod
370         from muneem.db_key import peek_key
371
372         db_path = db or load_config().db_path
373         key_state = "present in keychain" if peek_key() else "not yet minted"

```

```

374         if not Path(db_path).exists():
375             typer.echo(f"ledger : {db_path} (does not exist yet)")
376             typer.echo(f"key      : {key_state}")
377             return
378         if dbmod.is_plaintext_sqlite(db_path):
379             state = "PLAINTEXT ? run 'muneem db encrypt' to harden it"
380         else:
381             state = "encrypted (SQLCipher)"
382         typer.echo(f"ledger : {db_path}")
383         typer.echo(f"state  : {state}")
384         typer.echo(f"key    : {key_state}")
385
386     @db_app.command("encrypt")
387     def db_encrypt(
388         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
389 dir)."),
390     ) -> None:
391         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
392 place."""
393         from muneem import db as dbmod
394         from muneem.db_key import get_or_create_key
395
396         db_path = db or load_config().db_path
397         if not Path(db_path).exists():
398             typer.echo(
399                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
400 encrypted)."
401             )
402             return
403         if not dbmod.is_plaintext_sqlite(db_path):
404             typer.echo(
405                 f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
406 to do."
407             )
408             return
409         n = dbmod.encrypt_database(db_path, get_or_create_key())
410         typer.echo(
411             f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
412 remains at "
413             f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
414         )
415
416     def _require_encrypted_ledger(db_path: Path) -> str:
417         """Return the ledger's keychain key, or exit with a clear message if it can't be
418 used."""
419         from muneem import db as dbmod
420         from muneem.db_key import peek_key
421
422         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
423             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
424 err=True)

```

6. user (2026-06-10T16:28:40.188Z)

```

src\muneem\db.py:413: def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) ->
int:
src\muneem\db.py:443: def insert_au_transaction(conn: sqlite3.Connection, **fields: object) ->
int:
src\muneem\cli.py:134:         count = parser.parse_and_insert(pdf, conn, doc_id,
password=password)
src\muneem\cli.py:179:         count = parser.parse_and_insert(pdf, conn, doc_id,
password=password)
src\muneem\parsers\au.py:39:     def parse_and_insert(

```

```

src\muneem\parsers\au.py:61:         dbmod.insert_au_transaction(
src\muneem\parsers\axis.py:66:     def parse_and_insert(
src\muneem\parsers\axis.py:94:         dbmod.insert_axis_transaction(
src\muneem\parsers\axis.py:169:     def parse_and_insert(
src\muneem\parsers\base.py:31:     def parse_and_insert(
src\muneem\parsers\hdfc.py:416:     def parse_and_insert(

```

7. user (2026-06-10T16:28:50.200Z)

```

1      """AU Small Finance Bank savings / transaction-account statement parser.
2
3      Runs on the **shared engine-driven path** (:mod:`muneem.parsers.savings`): per-table +
4      coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
5      gated** so a
6      vintage it can't read stores nothing rather than mislabeled rows. The AU specifics are
7      entirely in
8      ``AUBANK_PROFILE`` ? this parser is mostly the DB wiring. No statement content is logged
9      here.
10
11     """
12     from __future__ import annotations
13
14     import sqlite3
15     from pathlib import Path
16
17     from muneem import __version__
18     from muneem import db as dbmod
19     from muneem.errors import ParseError
20     from muneem.redaction import redact_filename
21
22     from . import register_parser
23     from .base import StatementParser
24     from .profiles import AUBANK_PROFILE
25     from .savings import SavingsParse, parse_savings
26     from .validation import Confidence
27
28     def parse_au_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
29         """Parse an AU Bank savings statement via the shared engine-driven path."""
30         return parse_savings(pdf_path, AUBANK_PROFILE, password)
31
32     @register_parser
33     class AUParser(StatementParser):
34         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
35         None) -> bool:
36             if sender and "aubank" in sender.lower():
37                 return True
38             low = (text or "").lower()
39             return "au small finance bank" in low or "aubank" in low
40
41     def parse_and_insert(
42         self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
43         None = None
44     ) -> int:
45         """Persist a savings parse **only if it reconciles**; otherwise store
46         nothing."""
47         try:
48             parsed = parse_au_savings(pdf_path, password)
49         except ParseError:
50             raise
51         except Exception as exc:
52             raise ParseError(f"failed to parse AU statement
53 {redact_filename(pdf_path)}") from exc
54
55

```

```

50         if not parsed.verified:
51             dbmod.record_parse(
52                 conn,
53                 doc_id,
54                 confidence=Confidence.UNRECONCILED.value,
55                 parser="AUParser",
56                 parser_version=__version__,
57             )
58         return 0
59
60     for r in parsed.rows:
61         dbmod.insert_a_u_transaction(
62             conn,
63             document_id=doc_id,
64             account_number=parsed.account_number,
65             txn_date=r.txn_date,
66             value_date=r.value_date or "",
67             particulars=r.narration,
68             debit=r.debit,
69             credit=r.credit,
70             balance=r.balance,
71         )
72     dbmod.record_parse(
73         conn,
74         doc_id,
75         confidence=Confidence.RECONCILED.value,
76         parser="AUParser",
77         parser_version=__version__,
78     )
79     return len(parsed.rows)
80

```

8. user (2026-06-10T16:28:50.201Z)

```

1     """Axis Bank statement parsers: savings/transaction account + credit card.
2
3     **Savings runs on the shared engine-driven path** (:mod:`muneem.parsers.savings`): per-
table +
4     coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
gated** so a
5     vintage it can't read stores nothing (`unreconciled`) rather than mislabeled rows.
Axis specifics
6     live entirely in `AXIS_PROFILE`; this replaced the old fixed-column `extract_tables`
mapping.
7
8     Credit-card statements carry no running balance, so `AxisCreditCardParser` can't
reconcile
9     (parse/persist split + count only). No statement content is logged here.
10    """
11
12    from __future__ import annotations
13
14    import re
15    import sqlite3
16    from dataclasses import dataclass
17    from pathlib import Path
18
19    import pdfplumber
20
21    from muneem import __version__
22    from muneem import db as dbmod
23    from muneem.errors import ParseError
24    from muneem.redaction import redact_filename
25
26    from . import register_parser

```

```

27     from .base import StatementParser
28     from .profiles import AXIS_PROFILE
29     from .savings import SavingsParse, parse_savings, parse_statement_summary,
txn_line_predicate
30     from .validation import Confidence, parse_amount
31
32     _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}") # Axis credit card: dd/mm/yyyy
33     _CARD_RES = [re.compile(r"(\d{4}XXXX\d{4})"), re.compile(r"(\d{6}\*\d{4})")]
34
35
36     def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
37         for rx in patterns:
38             m = rx.search(text)
39             if m:
40                 return m.group(1).strip()
41         return None
42
43
44     # --- Savings / transaction account (concept core)
-----
45
46
47     # Back-compat names; the savings logic now lives in muneem.parsers.savings (shared with
AU Bank).
48     AxisSavings = SavingsParse
49     parse_axis_summary = parse_statement_summary
50     _is_txn_line = txn_line_predicate(AXIS_PROFILE)
51
52
53     def parse_axis_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
54         """Parse an Axis savings statement via the shared engine-driven path
(savings.parse_savings)."""
55         return parse_savings(pdf_path, AXIS_PROFILE, password)
56
57
58     @register_parser
59     class AxisParser(StatementParser):
60         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
61             if sender and "axisbank.com" in sender.lower():
62                 return True
63             t = text or ""
64             return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
65
66         def parse_and_insert(
67             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
68         ) -> int:
69             """Persist a savings parse **only if it reconciles**; otherwise store nothing.
70
71             Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
72             parser can't yet read surfaces loudly instead of saving mislabeled rows.
73             """
74             try:
75                 parsed = parse_axis_savings(pdf_path, password)
76             except ParseError:
77                 raise
78             except Exception as exc:
79                 # Filename only ? never statement text; original cause kept via `from`.
80                 name = redact_filename(pdf_path)
81                 raise ParseError(f"failed to parse Axis statement {name}") from exc
82
83             if not parsed.verified:

```

```

84         dbmod.record_parse(
85             conn,
86             doc_id,
87             confidence=Confidence.UNRECONCILED.value,
88             parser="AxisParser",
89             parser_version=__version__,
90         )
91     return 0
92
93     for r in parsed.rows:
94         dbmod.insert_axis_transaction(
95             conn,
96             document_id=doc_id,
97             account_number=parsed.account_number,
98             txn_date=r.txn_date,
99             value_date=r.value_date or "",
100             particulars=r.narration,
101             debit=r.debit,
102             credit=r.credit,
103             balance=r.balance,
104         )
105         dbmod.record_parse(
106             conn,
107             doc_id,
108             confidence=Confidence.RECONCILED.value,
109             parser="AxisParser",
110             parser_version=__version__,
111         )
112     return len(parsed.rows)
113
114
115     # --- Credit card (no running balance; count only)
116     -----
117
118     @dataclass
119     class AxisCCTxn:
120         txn_date: str
121         particulars: str
122         merchant_category: str
123         amount: float
124         type: str # "Cr" or "Dr"
125
126
127     def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
128         """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
129         rows: list[AxisCCTxn] = []
130         for table in tables or []:
131             for row in table or []:
132                 if not row or len(row) < 9:
133                     continue
134                 date_str = str(row[0]).strip() if row[0] else ""
135                 if not _CC_DATE_RE.match(date_str):
136                     continue
137                 amount_str = str(row[8]).strip() if row[8] else ""
138                 is_cr = "Cr" in amount_str
139                 amount = parse_amount(re.sub(r"^\d.", "", amount_str)) or 0.0
140                 rows.append(
141                     AxisCCTxn(
142                         txn_date=date_str,
143                         particulars=str(row[2]).strip() if row[2] else "",
144                         merchant_category=str(row[7]).strip() if row[7] else "",
145                         amount=amount,
146                         type="Cr" if is_cr else "Dr",
147                     )

```

```

148         )
149     return rows
150
151
152     def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisCCTxn]]:
153         """Open an Axis credit-card statement and return (card_number, transactions)."""
154         card_number = "UNKNOWN"
155         rows: list[AxisCCTxn] = []
156         with pdfplumber.open(pdf_path, password=password) as pdf:
157             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
158             card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
159             for page in pdf.pages:
160                 rows.extend(_rows_from_cc_tables(page.extract_tables()))
161         return card_number, rows
162
163
164     @register_parser
165     class AxisCreditCardParser(StatementParser):
166         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
167             return "Axis Bank" in text and "Credit Card Statement" in text
168
169         def parse_and_insert(
170             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
171         ) -> int:
172             try:
173                 card_number, rows = parse_axis_cc(pdf_path, password)
174             except ParseError:
175                 raise
176             except Exception as exc:
177                 raise ParseError(
178                     f"failed to parse Axis CC statement {redact_filename(pdf_path)}"
179                 ) from exc
180             for t in rows:
181                 dbmod.insert_axis_cc_transaction(
182                     conn,
183                     document_id=doc_id,
184                     account_number=card_number,
185                     txn_date=t.txn_date,
186                     particulars=t.particulars,
187                     merchant_category=t.merchant_category,
188                     amount=t.amount,
189                     type=t.type,
190                 )
191             # No running balance to reconcile; documents.status stays as the caller set it.
192             # record provenance (with no confidence verdict).
193             dbmod.record_provenance(
194                 conn,
195                 document_id=doc_id,
196                 source_type="pdf_statement",
197                 parser="AxisCreditCardParser",
198                 parser_version=__version__,
199                 mapping_layer="deterministic",
200             )
201             return len(rows)
202

```

9. assistant (2026-06-10T16:29:23.878Z)

The core code claims check out. Now let me see how the CLI surfaces the verdict at parse time, and check tests for whether this behavior is intentional.

10. user (2026-06-10T16:29:25.568Z)

```
80     def info() -> None:
81         """Show resolved local paths (no secrets, no sensitive data)."""
82         cfg = load_config()
83         present = "present" if cfg.config_file.is_file() else "absent"
84         typer.echo(f"muneem {__version__}")
85         typer.echo(f"config dir : {cfg.config_dir}")
86         typer.echo(f"data dir   : {cfg.data_dir}")
87         typer.echo(f"db path   : {cfg.db_path}")
88         typer.echo(f"staging   : {cfg.staging_dir}")
89         typer.echo(f"config file: {cfg.config_file} ({present})")
90
91
92     def _ingest_bank_statement(pdf: Path, db_path: Path, doc_type: str, *, interactive:
bool) -> None:
93         """Unlock (if encrypted) ? extract ? select parser ? store reconciled transactions.
94
95         The hybrid password unlock (cached/configured candidates ? prompt) runs *before*
extraction,
96         since the text needed to pick a parser is itself encrypted. The resolved password is
forwarded
97         to extraction and the parser, used in-memory only ? never logged.
98         """
99         from muneem import db as dbmod
100        from muneem.extract import extract
101        from muneem.parsers import get_parser
102        from muneem.statement_unlock import resolve_password
103        from muneem.unlock import is_encrypted
104
105        password: str | None = ""
106        if is_encrypted(pdf):
107            password = resolve_password(pdf, interactive=interactive)
108            if password is None:
109                hint = " (run without --no-prompt to enter it)" if not interactive else ""
110                typer.echo(f"Could not unlock {pdf.name}: no working password{hint}.",
err=True)
111                raise typer.Exit(1)
112
113        ex = extract(pdf, password=password or "") # resolved above; `or ""` keeps the type
str
114        parser = get_parser(ex.text)
115        if parser is None:
116            typer.echo(f"No parser recognizes {pdf.name} as a supported bank statement.",
err=True)
117            raise typer.Exit(1)
118
119        with _ledger(db_path) as conn:
120            dbmod.init_db(conn)
121            existing = dbmod.get_document_by_sha(conn, ex.sha256)
122            if existing is not None:
123                typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
124                return
125            doc_id = dbmod.insert_document(
126                conn,
127                source="file",
128                external_id=pdf.name,
129                doc_type=doc_type,
130                sha256=ex.sha256,
131                path=str(pdf.resolve()),
132                status="parsed",
133            )
134            count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
135            doc = dbmod.get_document_by_sha(conn, ex.sha256)
136            status = doc["status"] if doc else "parsed"
```

```

137
138     typer.echo(
139         f"ingested {pdf.name} (doc {doc_id}); parsed {count} transaction(s) "
140         f"via {parser.__class__.__name__} [{status}]"
141     )
142
143
144 def _eval_statement(pdf: Path, *, interactive: bool) -> tuple[str, int, str | None]:
145     """Run one statement through the full parse pipeline into an in-memory DB.
146
147     Returns ``(outcome, row_count, parser_name)`` where ``outcome`` is the document
148     status
149     (``reconciled`` / ``unreconciled``) or a failure tag (``locked`` / ``no-parser`` /
150     ``error``).
151     Nothing is persisted and no statement value is surfaced ? only the verdict and
152     counts.
153     """
154     from muneem import db as dbmod
155     from muneem.extract import extract
156     from muneem.parsers import get_parser
157     from muneem.statement_unlock import resolve_password
158     from muneem.unlock import is_encrypted
159
160     password = ""
161     if is_encrypted(pdf):
162         password = resolve_password(pdf, interactive=interactive) or ""
163         if not password:
164             return ("locked", 0, None)
165     try:
166         ex = extract(pdf, password=password)
167         parser = get_parser(ex.text)
168     except Exception as exc: # eval must not crash on one bad file; record the redacted
169         reason
170         _log.warning(
171             "eval: extract/route failed for %s (%s)", redact_filename(pdf),
172             type(exc).__name__
173         )
174         return ("error", 0, None)
175     if parser is None:
176         return ("no-parser", 0, None)
177     name = parser.__class__.__name__
178     try:
179         with dbmod.connect(":memory:") as conn:
180             dbmod.init_db(conn)
181             doc_id = dbmod.insert_document(
182                 conn, source="file", external_id=pdf.name, sha256=ex.sha256,
183                 status="parsed"
184             )
185             count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
186             doc = dbmod.get_document_by_sha(conn, ex.sha256)
187             status = doc["status"] if doc else "parsed"
188     except Exception as exc:
189         _log.warning(
190             "eval: parse/store failed for %s (%s)", redact_filename(pdf),
191             type(exc).__name__
192         )
193         return ("error", 0, name)
194     return (status, count, name)
195
196 @app.command("eval")
197 def eval_statements(
198     folder: Path = typer.Argument(
199         ...,
200         exists=True,

```

```

195         file_okay=False,
196         readable=True,
197         help="A LOCAL folder of statement PDFs to evaluate (e.g. your seed or golden
set).",
198     ),
199     prompt: bool = typer.Option(
200         False,
201         "--prompt/--no-prompt",
202         help="Prompt for a password when cached/configured candidates don't unlock a
file.",
203     ),
204 ) -> None:
205     """Evaluate the parsers against a folder of statements and report the reconcile
rate.
206
207     The seed?golden onboarding loop (see docs/onboarding-a-bank.md): point it at a few
*seed*
208     statements while tuning a profile, then at a *golden* set to confirm. Everything
runs locally in
209     an in-memory DB; only counts / statuses / parser names print ? never statement
values.
210     """
211     pdfs = sorted(folder.glob("*.pdf"))
212     if not pdfs:
213         typer.echo(f"no .pdf files in {folder}")
214         return
215     reconciled = 0
216     for pdf in pdfs:
217         outcome, count, name = _eval_statement(pdf, interactive=prompt)
218         reconciled += outcome == "reconciled"
219         typer.echo(f" {outcome:12} {count:4d} row(s) {name or '?':20} {pdf.name}")

```

11. user (2026-06-10T16:29:26.476Z)

```

docs\CURRENT_STATE.md:29:[Omitted long matching line]
docs\CURRENT_STATE.md:30:[Omitted long matching line]
docs\onboarding-a-bank.md:28:It prints, per file, the outcome (`reconciled` / `unreconciled` /
`locked` / `no-parser` / `error`),
docs\onboarding-a-bank.md:34:> mislabeled rows. Diagnose with the (masked) clustering tools; the
`unreconciled` files are safe.
docs\ingesting-statements.md:27:`[reconciled]` means the parse is trusted; `[unreconciled]`
means it couldn't be verified, so
docs\ingesting-statements.md:89:| `? [unreconciled]` (0 transactions) | The parser couldn't
verify the parse (e.g. an Axis vintage `extract_tables` mangles); nothing was stored. See
[parser-architecture.md](./parser-architecture.md). |
docs\parser-architecture.md:111: flagged; the oracle-gated parsers already refuse to persist
unreconciled rows)
docs\schema.md:44:| status | TEXT | NOT NULL default `parsed`; parsers set `reconciled` /
`unreconciled` |
src\muneem\cli.py:148:    (`reconciled` / `unreconciled`) or a failure tag (`locked` /
`no-parser` / `error`).
tests\parsers\test_au.py:71:         assert dbmod.get_document_by_sha(conn, "au2")["status"] ==
"unreconciled"
tests\parsers\test_axis.py:76:def test_savings_refuses_when_unreconciled():
tests\parsers\test_axis.py:114:         assert dbmod.get_document_by_sha(conn, "ax2")["status"]
== "unreconciled"
tests\parsers\test_hdfc.py:197:def test_parse_and_insert_records_unreconciled(tmp_path,
monkeypatch):
tests\parsers\test_hdfc.py:210:         assert dbmod.get_document_by_sha(conn, "h2")["status"] ==
"unreconciled"
tests\parsers\test_local_au.py:24:def test_local_au_never_stores_unreconciled(pdf_path,
tmp_path):
tests\parsers\test_local_au.py:47:         f"{pdf_path.name}: stored {n} row(s) with
status={status} ? must not persist unreconciled"
tests\parsers\test_local_axis.py:26:def test_local_axis_never_stores_unreconciled(pdf_path,

```

```

tmp_path):
tests\parsers\test_local_axis.py:45:      # document unreconciled) or, if it stored rows, they
reconciled. Never garbage.
tests\parsers\test_local_axis.py:49:      "stop-gap must not persist unreconciled rows"
src\muneem\parsers\au.py:54:      confidence=Confidence.UNRECONCILED.value,
tests\test_provenance.py:75: def
test_unreconciled_parse_records_unreconciled_provenance(monkeypatch):
tests\test_provenance.py:82:      assert dbmod.get_provenance(conn, doc)["confidence"] ==
"unreconciled"
tests\parsers\test_validation.py:94:      assert not r.ok and r.confidence is
Confidence.UNRECONCILED
src\muneem\parsers\axis.py:5: vintage it can't read stores nothing (`unreconciled`) rather than
mislabeled rows. Axis specifics
src\muneem\parsers\axis.py:71:      Sets ``documents.status`` to ``reconciled`` /
``unreconciled`` accordingly, so a layout the
src\muneem\parsers\axis.py:87:      confidence=Confidence.UNRECONCILED.value,
src\muneem\parsers\hdfc.py:425:      Sets ``documents.status`` to 'reconciled' /
'unreconciled' based on the
src\muneem\parsers\hdfc.py:454:      verdict = Confidence.RECONCILED if ok else
Confidence.UNRECONCILED
src\muneem\parsers\validation.py:124:      ``RECONCILED`` ? the parse passed every available
check, so it is trusted. ``UNRECONCILED`` ?
src\muneem\parsers\validation.py:129:      UNRECONCILED = "unreconciled"
src\muneem\parsers\validation.py:154:      return Confidence.RECONCILED if self.ok else
Confidence.UNRECONCILED

```

12. user (2026-06-10T16:29:40.569Z)

```

150         date="05/02/2026",
151         narration="UPI-SWIGGY",
152         withdrawal=500.0,
153         deposit=0.0,
154         closing_balance=9500.0,
155         value_date="04/02/2026",
156         ref="123456789012",
157         page=3,
158     )
159     c = t.to_concept()
160     assert c.txn_date == "05/02/2026" and c.narration == "UPI-SWIGGY"
161     assert c.debit == 500.0 and c.credit == 0.0 and c.balance == 9500.0
162     assert c.value_date == "04/02/2026" and c.reference == "123456789012"
163
164
165     def test_can_handle():
166         parser = hdfc.HDFCParser()
167         statement = "HDFC BANK Savings Account Details Narration Closing Balance"
168         assert parser.can_handle(statement)
169         assert parser.can_handle(statement, sender="alerts@hdfcbank.net")
170         # sender alone, with no statement markers, must NOT match (e.g. a promo from HDFC)
171         assert not parser.can_handle("anything", sender="alerts@hdfcbank.net")
172         assert not parser.can_handle("HDFC Bank promo: use code SAVE50")
173         assert not parser.can_handle("Axis Bank Statement of Account Narration Closing
Balance")
174
175
176     def test_parse_and_insert_records_reconciled(tmp_path, monkeypatch):
177         monkeypatch.setattr(hdfc, "parse_statement", lambda *a, **k: _sample_statement())
178         monkeypatch.setattr(
179             hdfc,
180             "parse_summary",
181             lambda *a, **k: hdfc.StatementSummary(
182                 opening_balance=10000.0, closing_balance=9999.50, debit_count=2,
credit_count=1
183             ),
184         )

```

```

185         with dbmod.connect(tmp_path / "t.db") as conn:
186             dbmod.init_db(conn)
187             doc_id = dbmod.insert_document(conn, source="file", sha256="h1",
status="parsed")
188             n = hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
189             assert n == 3
190             rows = dbmod.list_hdfc_transactions(conn, limit=100)
191             assert len(rows) == 3
192             assert rows[0]["account_number"] == "50100000000000"
193             assert {r["particulars"] for r in rows} # narration mapped to particulars
194             assert dbmod.get_document_by_sha(conn, "h1")["status"] == "reconciled"
195
196
197     def test_parse_and_insert_records_unreconciled(tmp_path, monkeypatch):
198         broken = _parse_sample()
199         broken[2].closing_balance = 1.0 # break the balance walk
200         monkeypatch.setattr(
201             hdfc,
202             "parse_statement",
203             lambda *a, **k: hdfc.Statement(path=Path("x"), transactions=broken),
204         )
205         monkeypatch.setattr(hdfc, "parse_summary", lambda *a, **k: None) # fall back to
reconcile()
206         with dbmod.connect(tmp_path / "t.db") as conn:
207             dbmod.init_db(conn)
208             doc_id = dbmod.insert_document(conn, source="file", sha256="h2",
status="parsed")
209             hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
210             assert dbmod.get_document_by_sha(conn, "h2")["status"] == "unreconciled"
211
212
213     @pytest.mark.local_statements
214     @pytest.mark.parametrize(
215         "pdf_path",
216         sorted(Path("testdata/hdfc-statements").glob("*.pdf")),
217         ids=lambda p: p.name,
218     )
219     def test_real_hdfc_statement_validates(pdf_path, tmp_path):
220         """Local-only: parse the real statements and require the parse to agree with
221         HDFC's own SUMMARY (Dr/Cr counts + opening/closing) and reconcile fully.
222
223         Set MUNEEM_TEST_HDFC_PW; never runs in CI. Surfaces counts/booleans only.
224         """
225         pw = os.environ.get("MUNEEM_TEST_HDFC_PW")
226         if not pw:
227             pytest.skip("needs MUNEEM_TEST_HDFC_PW + real statements (never in CI)")
228
229         stmt = hdfc.parse_statement(pdf_path, password=pw)

```

13. user (2026-06-10T16:29:41.588Z)

```

85     - **Per-row walk:** `balance[i] == balance[i-1] ? debit[i] + credit[i]` (catches a
missed/misread row
86     *between* any two balances).
87     - **Total reconcile:** `opening + ?credits ? ?debits == closing` (works even when
there's no per-row
88     balance, e.g. Axis savings).
89     - **Bookend:** parsed Dr/Cr counts + opening/closing vs the bank's printed SUMMARY
(catches a missing
90     first/last row ? HDFC has this; others where available).
91     - **Selection:** among L0/L1/L2 candidate mappings, pick the reconciling one; if several
reconcile,
92     prefer the highest-confidence (header-backed) one.
93     - **Confidence verdict:** `reconciled` (per-row or total) + `bookended` ? trust; else
quarantine.

```

```

94
95     ## 6. Bank knowledge as *data* (profiles), not code
96
97     A bank profile is a small **declarative** object (likely YAML/dict): concept?label
aliases, accepted
98     date formats, anchor strings to locate the table, account-number patterns, known quirks
(multi-account
99     sections, "Other Information" column), and the password rule. **Generic detectors run
first; the
100     profile supplies priors/hints** to disambiguate ? it is not bespoke parsing logic.
Absorbing a small
101     format change = tweak a profile (often nothing ? the generic path copes). A brand-new
issuer = add a
102     profile, and only drop to custom code if the generic ladder genuinely can't cope.
103
104     ## 7. Failsafes ? the "traps" (loud drift, never silent)
105
106     The Axis lesson: a broken parse must *announce itself*. Built-in tripwires, each ? flag
+ provenance,
107     never store-and-pretend:
108
109     - header label matches no lexicon entry ? "unknown field, mapped by structure ? review"
110     - reconciliation breaks, or parsed counts ? printed summary ? **quarantine** (store
nothing or store
111     flagged; the oracle-gated parsers already refuse to persist unreconciled rows)
112     - a profile's expected anchor is missing ? "format may have changed for <bank>"
113     - _(L3)_ VLM output won't reconcile, or disagrees with a deterministic parse ? reject
the VLM
114     - **provenance + confidence on every parsed statement:** which layer/mapping produced
it, did it
115     reconcile / bookend. Trust is *recorded*, not assumed.
116
117     _Research (done):_ VLMs' top failure modes on financial numbers are **digit
transposition, decimal/
118     sign errors, dropped/duplicated rows, and column misalignment** ? which almost always
**break the
119     reconcile identity**, so the verifier rejects them. L3 traps therefore: gate every VLM
parse on
120     reconciliation; **self-consistency** re-sample (keep the pass that reconciles); use the
per-row walk
121     to localize a failing row for re-query or human review; cloud stays off unless
explicitly approved.
122
123     ## 8. Two tiers of "semantic"
124
125     - **Tier 1 ? field semantics** (this doc): what each column *means*. The resilience win.
Build first.
126     - **Tier 2 ? transaction semantics** (later): classify each row (UPI / NEFT / ATM /
charge / interest
127     / sweep), extract merchant/counterparty, assign category. Same schema, layered on top;
ties to the
128     parked merchant-canonicalization decision (DESIGN § 8.5).
129
130     ## 9. Data-model evolution
131
132     Today: per-bank raw tables (`hdfc_transactions`, `axis_transactions`, ?) ? fine for
fidelity, but the
133     concept schema points at a **unified `transactions` table** keyed on the canonical
concepts, plus
134     `account`/`document` context and **`provenance` + `confidence`** columns. Migration is
additive (the

```

14. user (2026-06-10T16:29:41.876Z)

```

1     # Ingesting bank statements (with password unlock)

```

```

2
3     `muneem ingest` reads a bank-statement PDF, **unlocks it if it's password-protected**,
parses it
4 into normalized transactions, and stores only a parse that **reconciles** (the running
balance
5 walks, or the totals match the printed opening/closing). Supported issuers today:
**HDFC, Axis, AU
6 Bank**.
7
8     ```bash
9     muneem ingest /path/to/statement.pdf --doc-type bank_statement
10    ```
11
12    If the PDF is encrypted, muneem resolves the password with a **hybrid strategy** (no
password is ever
13 written to the repo or logged):
14
15    1. **Cached candidates** ? passwords from earlier successful unlocks, stored in your
**OS keychain**.
16    2. **Configured candidates** ? entries in `password_rules.yaml` (see below).
17    3. **Interactive prompt** ? a hidden prompt (`getpass`). A password entered here that
works is
18     **cached in the keychain**, so the next ingest of any statement with the same
password is
19     friction-free.
20
21    On success you'll see, e.g.:
22
23    ```
24    ingested statement.pdf (doc 1); parsed 37 transaction(s) via HDFCParser [reconciled]
25    ```
26
27    `[reconciled]` means the parse is trusted; `[unreconciled]` means it couldn't be
verified, so
28    **nothing was stored** (muneem never persists mislabeled rows). Read parsed rows back
with
29    `muneem txns list hdfc` (or `axis` / `au`).
30
31    ## Enable unattended unlocking (recommended)
32
33    For batch/backfill or `--no-prompt` runs, put your statement passwords in a
**gitignored** rules
34    file so you're never prompted.
35
36    1. Find your config directory:
37
38    ```bash
39    muneem info          # ? "config dir : /home/you/.config/muneem" (varies by OS)
40    ```
41
42    2. Create `/password_rules.yaml`. **Never commit it** ? `.gitignore` already
blocks
43    `password_rules.*`, and it lives in your OS config dir (outside the repo) anyway.
44
45    ```yaml
46    # <config-dir>/password_rules.yaml  ?  NEVER commit this file.
47    # Quote every password (keeps leading zeros; avoids YAML parsing "0106" as a number).
48
49    # Per-issuer passwords:
50    hdfc: "<your-hdfc-pdf-password>"
51    axis: "<your-axis-pdf-password>"
52    aubank: "<your-au-pdf-password>"
53
54    # ?or a bank-agnostic list of passwords to try (handy when the issuer label varies):
55    candidates:

```

```

56         - "<password-a>"
57         - "<password-b>"
58     ```
59
60     muneem tries **all** of these (it has to pick the password before it can read the PDF
to know
61     the issuer), so either form works. A legacy `./password_rules.yaml` in the current
directory is
62     still read but warns ? prefer the config dir.
63
64     3. Ingest without prompts:
65
66     ```bash
67     muneem ingest statement.pdf --doc-type bank_statement --no-prompt
68     ```
69
70     `--no-prompt` uses only cached + configured candidates and **never** opens an
interactive prompt
71     (so it's safe in cron/CI). Without it, an unresolved password falls back to the
hidden prompt.
72
73     ## Security
74
75     - **Nothing sensitive is committed or logged.** `password_rules.*` is gitignored;
resolved passwords
76     live only in memory and the **OS keychain** (Windows Credential Manager / macOS
Keychain / Linux
77     Secret Service), via `muneem.secrets`.
78     - **No decrypted copy is written to disk** ? the password is handed straight to the text
extractor
79     and the parser in memory.
80     - Transaction parses are **reconciliation-gated**: an unverifiable parse stores nothing
rather than
81     risk mislabeled rows.
82
83     ## Troubleshooting
84
85     | Message | Meaning / fix |
86     |---|---|
87     | `Could not unlock <file>: no working password` | None of the cached/configured
candidates worked. Run without `--no-prompt` to type it, or add it to `password_rules.yaml`. |
88     | `No parser recognizes <file> as a supported bank statement` | The issuer isn't
supported yet (today: HDFC, Axis, AU Bank), or the text didn't match. |
89     | `'? [unreconciled]' (0 transactions)` | The parser couldn't verify the parse (e.g. an
Axis vintage `extract_tables` mangles); nothing was stored. See [ `parser-
architecture.md`](./parser-architecture.md). |
90
91     ## Headless Linux note
92
93     Keychain storage needs a backend. Use a desktop keyring (GNOME Keyring / KWallet), or
94     `pip install keyrings.alt` for a file-based backend. Until then, `password_rules.yaml` +
`--no-prompt`
95     works without any keychain (you just won't get prompt-caching).
96

```

15. assistant (2026-06-10T16:31:04.201Z)

Let me confirm BANK_DISPATCH and check the reconciler's behavior for completeness.

16. user (2026-06-10T16:31:05.731Z)

```

52: BANK_DISPATCH = {
53:     "axis": (dbmod.list_axis_transactions, "debit", "credit"),
54:     "axis_cc": (dbmod.list_axis_cc_transactions, "amount", "type"),
55:     "au": (dbmod.list_au_transactions, "debit", "credit"),

```

```

56-     "hdfc": (dbmod.list_hdfc_transactions, "withdrawal", "deposit"),
57-}
58-
59-
60-def _version_callback(value: bool) -> None:
--
343:     if bank not in BANK_DISPATCH:
344-         typer.echo(f"Unknown bank: {bank}", err=True)
345-         return
346-
347:     list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
348-     with _ledger(db_path) as conn:
349-         dbmod.init_db(conn)
350-         rows = list_fn(conn, limit=limit)
351-
352-     if not rows:
353-         typer.echo(f"no {bank} transactions found.")
354-         return
355-     for row in rows:

```

17. user (2026-06-10T16:31:06.773Z)

```

120
121     class Confidence(StrEnum):
122         """Recorded trust verdict for a parsed statement (mirrors ``documents.status``
values).
123
124         ``RECONCILED`` ? the parse passed every available check, so it is trusted.
``UNRECONCILED`` ?
125         a check failed (or none could run); the parse is suspect and is quarantined rather
than trusted.
126         """
127
128         RECONCILED = "reconciled"
129         UNRECONCILED = "unreconciled"
130
131
132     @dataclass
133     class BookendReport:
134         """A parse cross-checked against the bank's own SUMMARY. Booleans + counts only.
135
136         ``None`` for a check means the figure wasn't available to verify (not a failure).
``ok``
137         requires the per-row balance walk *plus* every available check to pass.
138         """
139
140         n: int
141         reconciles: bool
142         debit_count_ok: bool | None = None
143         credit_count_ok: bool | None = None
144         opening_ok: bool | None = None
145         closing_ok: bool | None = None
146
147         @property
148         def ok(self) -> bool:
149             checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok,
self.closing_ok]
150             return self.n > 0 and self.reconciles and all(c is True for c in checks)
151
152         @property
153         def confidence(self) -> Confidence:
154             return Confidence.RECONCILED if self.ok else Confidence.UNRECONCILED
155
156
157     def verify(

```

```

158         rows: Sequence[ConceptRow], concepts: StatementConcepts, tol: float = 0.01
159     ) -> BookendReport:
160         """Bookend a parse against the bank's printed SUMMARY: reconciliation + the printed
161         figures.
162         The per-row walk catches any mid-statement error/omission; the Dr/Cr counts catch
163         any
164         extra/missing row; the opening & closing balances catch a missing first/last row.
165         Each printed
166         figure is only checked when present (``None`` ? not checked). The opening balance is
167         checked by
168         implication (``balance[0] ? credit[0] + debit[0]`` should equal the printed
169         opening).
170         """
171         reconciles = reconcile_running_balance(_row_entries(rows), tol).ok
172         debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
173         if concepts.debit_count is not None:
174             debit_count_ok = sum(1 for r in rows if (r.debit or 0.0) > 0) ==
175             concepts.debit_count
176         if concepts.credit_count is not None:
177             credit_count_ok = sum(1 for r in rows if (r.credit or 0.0) > 0) ==
178             concepts.credit_count
179         if rows and rows[-1].balance is not None and concepts.closing_balance is not None:
180             closing_ok = abs(rows[-1].balance - concepts.closing_balance) <= tol
181         if rows and rows[0].balance is not None and concepts.opening_balance is not None:
182             implied_opening = rows[0].balance - (rows[0].credit or 0.0) + (rows[0].debit or
183             0.0)
184             opening_ok = abs(implied_opening - concepts.opening_balance) <= tol
185         return BookendReport(
186             n=len(rows),

```

18. user (2026-06-10T16:31:45.656Z)

Structured output provided successfully

19. assistant (2026-06-10T16:31:53.623Z)

Verification complete. The finding is real: HDFC persists all rows on an unreconciled parse (unlike the gated Axis/AU parsers), the only read path serves them unflagged, the docs falsely promise nothing was stored, and sha-dedup prevents clean re-ingestion. The tangential closing_balance=0.0 claim was refuted as dead code. Severity high upheld at 0.85 confidence.